

Module-1

What is web Framework?

A web framework is a software tool that helps developers build websites and web applications easily.

It provides ready-made components for handling URLs, database connection, forms, and user authentication, which saves time and reduces errors.

Introduction to Django

Django is a high-level, open-source web framework written in **Python** that enables developers to build secure, scalable, and maintainable web applications quickly. It was designed to promote rapid development and clean, practical design.

Django follows the **MVT (Model–View–Template)** architecture:

- **Model** – Handles database and data structure
- **View** – Contains business logic and processes user requests
- **Template** – Manages presentation layer (HTML)

Key Features of Django

1. **Built-in Admin Panel** – Automatically generated admin interface to manage database records.
2. **Authentication System** – Provides login, logout, password management, and user roles.
3. **ORM (Object Relational Mapper)** – Allows interaction with the database using Python code instead of SQL.
4. **URL Routing** – Maps URLs to views easily.
5. **Form Handling** – Simplifies form creation and validation.
6. **Security** – Protects against common attacks like SQL injection, CSRF, and XSS.

7. **Scalability** – Suitable for small to large-scale applications.

Why Django is Called “Batteries-Included”

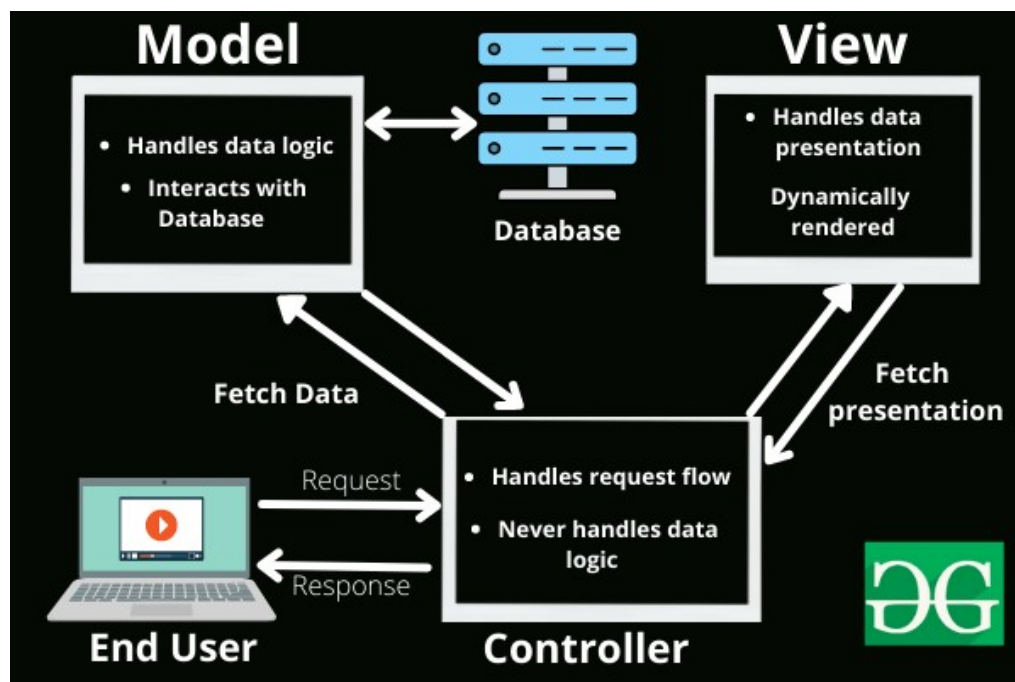
Django is known as a “**batteries-included**” framework because it provides most of the required tools (authentication, admin, ORM, security, etc.) by default. Developers do not need to install many external libraries.

Applications of Django

Django is widely used for:

- Dynamic websites
- E-commerce applications
- Content management systems
- Database-driven web applications

MVC Architecture (Model–View–Controller)



The **Model–View–Controller** framework is an architectural/design pattern that separates an application into three main logical components **Model**, **View**, and **Controller**. Each architectural component is built to handle specific development aspects of an application. It

isolates the business logic and presentation layer from each other. It was traditionally used for desktop **graphical user interfaces (GUIs)**. Nowadays, MVC is one of the most frequently used industry-standard web development frameworks to create scalable and extensible projects. It is also used for designing mobile apps.

The main goal of this design pattern was to solve the problem of users controlling a large and complex data set by splitting a large application into specific sections that all have their own purpose.

Features of MVC

- It provides a clear separation of **business logic, UI logic, and input logic**.

- It offers full control over your HTML and URLs which makes it easy to design web application architecture.

- It is a powerful URL-mapping component using which we can build applications that have comprehensible and searchable URLs.

- It supports **Test Driven Development (TDD)**.

Components of MVC

The MVC framework includes the following 3 components:

- Controller

- Model

- View

Controller:

The controller is the component that enables the interconnection between the views and the model so it acts as an intermediary. The controller doesn't have to worry about handling data logic, it just tells the model what to do. It processes all the business logic and incoming requests, manipulates data using the **Model** component, and interact with the **View** to render the final output.

Responsibilities:

- Receiving user input and interpreting it.

- Updating the Model based on user actions.

- Selecting and displaying the appropriate View.

Example: In a bookstore application, the Controller would handle actions such as searching for a book, adding a book to the cart, or checking out.

View:

The **View** component is used for all the UI logic of the application. It generates a user interface for the user. Views are created by the data which is collected by the model component but these data aren't taken directly but through the controller. It only interacts with the controller.

Responsibilities:

- Rendering data to the user in a specific format.
- Displaying the user interface elements.
- Updating the display when the Model changes.

Example: In a bookstore application, the View would display the list of books, book details, and provide input fields for searching or filtering books.

Model:

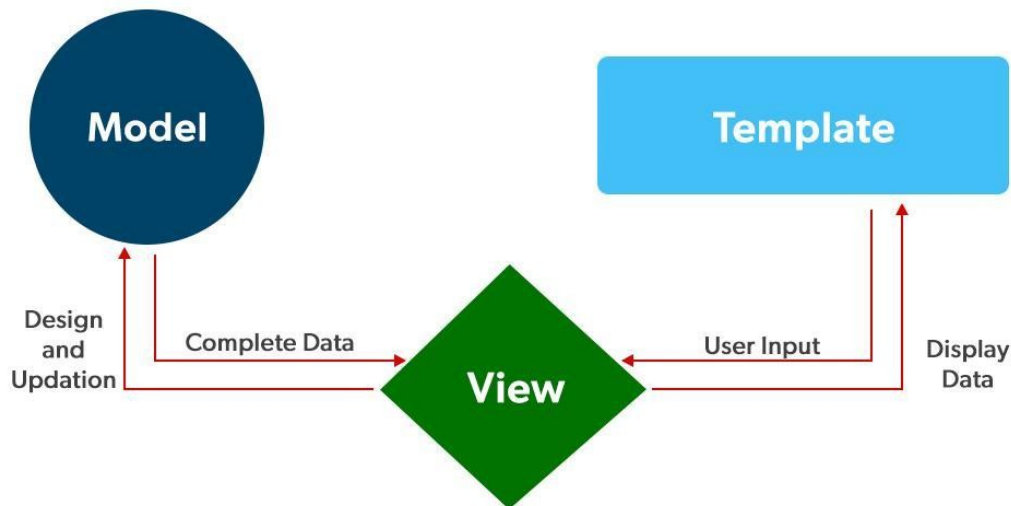
The **Model** component corresponds to all the data-related logic that the user works with. This can represent either the data that is being transferred between the View and Controller components or any other business logic-related data. It can add or retrieve data from the database. It responds to the controller's request because the controller can't interact with the database by itself. The model interacts with the database and gives the required data back to the controller.

Responsibilities:

- Managing data: CRUD (Create, Read, Update, Delete) operations.
- Enforcing business rules.
- Notifying the View and Controller of state changes.

Example: In a bookstore application, the Model would handle data related to books, such as the book title, author, price, and stock level

MVT (Model–View–Template)



The **Model–View–Template (MVT)** framework is an architectural/design pattern used mainly in Django. It separates an application into three main logical components: **Model, View, and Template**.

Each component handles a specific development aspect of the application. Like MVC, MVT isolates business logic from the presentation layer, making applications organized, scalable, and easy to maintain.

The main goal of MVT is to divide a large application into smaller sections with clear responsibilities. In Django, much of the controller-like work is handled internally by the framework, which simplifies development.

Features of MVT

- Clear separation of business logic, UI logic, and request handling
- Built-in URL routing system
- Automatic admin interface
- Powerful ORM for database operations
- Supports rapid development
- Secure and scalable architecture

Components of MVT

The MVT framework includes the following 3 components:

- Model
- View
- Template

Model

The **Model** component handles all data-related logic of the application. It defines the structure of the database tables and manages interactions with the database using Django's ORM.

The Model does not deal with how the data is displayed. It only focuses on storing, retrieving, and manipulating data.

Responsibilities:

- Managing data (CRUD operations)
- Defining database structure
- Enforcing business rules
- Communicating with the database

Example:

In a bookstore application, the Model would handle book details such as title, author, price, and stock quantity.

View

The **View** component contains the application's business logic and handles HTTP requests and responses. It receives user requests, interacts with the Model if needed, and sends data to the Template for presentation.

Unlike MVC, where Controller handles requests, in MVT the View performs that role.

Responsibilities:

By: Rakshitha P, Asst.Prof,Department of Computer Applications, NMAMIT, Nitte

- Receiving and processing user requests
- Interacting with the Model to fetch or update data
- Sending data to the Template
- Returning HTTP responses

Example:

In a bookstore application, the View would handle searching for books, adding books to cart, or displaying book details.

Template

The **Template** component is responsible for the presentation layer (UI). It defines how data is displayed to the user. Templates are usually HTML files containing Django Template Language for rendering dynamic content.

The Template does not contain business logic; it only displays the data passed from the View.

Responsibilities:

- Rendering data in a specific format (HTML)
- Displaying user interface elements
- Presenting dynamic content

Example:

In a bookstore application, the Template would display the list of books, book details page, and search form.

Creating and Importance of Virtual Environments in VS Code

A virtual environment is an isolated Python environment that allows you to install packages and dependencies separately for each project.

It ensures that one project's libraries do not affect another project.

Creating a Virtual Environment in VS Code

Step 1: Install Python

Make sure Python is installed on your system.

Step 2: Open Project Folder in VS Code

1. Open VS Code
2. Click File → Open Folder
3. Select your project folder

Step 3: Open Terminal in VS Code

Go

to:

Terminal → New Terminal

Step 4: Create Virtual Environment

Run the following command:

```
python -m venv env
```

Here:

- **venv** = module used to create virtual environment
- **env** = folder name (you can give any name)

This will create a folder named env inside your project.

Step 5: Activate Virtual Environment

```
env\Scripts\activate
```

After activation, you will see:

(env)

In the terminal.

Importance of Virtual Environment in VS Code

Dependency Isolation

A virtual environment allows each project to have its own separate set of Python packages. This means different projects can use different versions of the same library without affecting each other. It prevents conflicts and ensures smooth project development.

Avoids Version Conflicts

Sometimes one project may require an older version of a package while another project requires the latest version. A virtual environment keeps these dependencies separate, avoiding errors and compatibility issues.

Keeps System Python Clean

Without a virtual environment, packages are installed globally in the system Python, which can make the system messy and unstable. Using a virtual environment ensures that project-related libraries are stored only inside the project folder.

Easy Project Sharing and Deployment

Virtual environments help in generating a `requirements.txt` file, which lists all installed packages. This makes it easy to share the project with others or deploy it on another system by installing the same dependencies.

Professional and Industry Standard Practice

Using virtual environments is considered a best practice in Python development. It improves project organization, maintainability, and scalability, especially when working in teams or on large projects.

Safe Experimentation

Developers can test new libraries or features inside a virtual environment without worrying about breaking other projects or the main system configuration.

Installing and Setting Up a Django Project

Step 1: Install Python

1. Download Python from the official website.
2. During installation, **tick “Add Python to PATH”**.
3. Verify installation in Command Prompt:

```
python --version
```

Step 2: Create a Virtual Environment (Recommended)

Open Command Prompt and go to your project folder:

```
cd Desktop
```

Create virtual environment:

```
python -m venv env
```

Activate it:

```
env\Scripts\activate
```

After activation, you will see (env) in the command prompt.

Step 3: Install Django

```
pip install django
```

Check Django version:

```
django-admin --version
```

Step 4: Create a Django Project

```
django-admin startproject myproject
```

Move into the project folder:

```
cd myproject
```

Step 5: Run the Development Server

```
python manage.py runserver
```

Open browser and type:

<http://127.0.0.1:8000/>

You will see the Django welcome page.

Step 6: Create an App

```
python manage.py startapp myapp
```

Then add 'myapp' inside INSTALLED_APPS in [settings.py](#).

Project Structure of a Django Project

When you create a Django project using:

```
django-admin startproject myproject
```

The folder structure will look like this:

```
myproject/
|
|—— manage.py
|
|—— myproject/
    |—— __init__.py
    |—— settings.py
    |—— urls.py
    |—— asgi.py
    |—— wsgi.py
```

manage.py

It is a command-line utility used to interact with the project.
It helps in running the server, migrations, creating apps, etc.

```
python manage.py runserver
```

myproject/ (Inner Folder)

This is the main project configuration folder.

init.py

This file tells Python that this folder is a Python package.

settings.py

settings.py is the main configuration file of the Django project. It controls how your project behaves.

Important Sections Inside settings.py:

DEBUG

```
DEBUG = True
```

Used during development.

Shows detailed error pages.

Should be **False** in production.

ALLOWED_HOSTS

```
ALLOWED_HOSTS = []
```

- Specifies which domains can access your project.
- Required when `DEBUG = False`.

INSTALLED_APPS

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'myapp',  
]
```

- List of apps used in the project.
- You must add your app name here after creating it.

DATABASES

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.sqlite3',  
        'NAME': BASE_DIR / 'db.sqlite3',  
    }  
}
```

Defines database configuration.

By default, Django uses SQLite.

MIDDLEWARE

- Handles request and response processing.

- Used for security, sessions, authentication, etc.

TEMPLATES

- Defines template folder location.
- Controls HTML rendering settings.

STATICFILES

- Used for CSS, JavaScript, images.
- Configures static file handling.

urls.py

urls.py is used for URL routing. It connects URLs to views.

```
from django.contrib import admin
from django.urls import path
from myapp import views
```

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path("", views.home),
]
```

Purpose:

- Maps URL patterns to functions.
- When a user types a URL, Django checks this file.
- Directs request to the correct view.

wsgi.py

Used for deploying the project in production servers.
It helps web servers communicate with Django.

asgi.py

Used for asynchronous support (real-time features like chat applications).

Creating App

If you run:

By: Rakshitha P, Asst.Prof, Department of Computer Applications, NMAMIT, Nitte

python manage.py startapp myapp

You will get:

```
myapp/
|____ migrations/
|____ __init__.py
|____ admin.py
|____ apps.py
|____ models.py
|____ tests.py
|____ views.py
```

In Django, each app contains several important files that handle specific responsibilities. These files help organize the application in a structured and modular way.

models.py is used to define the database structure of the application. In this file, we create classes that represent database tables. Each class is called a model, and each attribute of the class represents a field (column) in the table. Django automatically converts these models into database tables using migrations.

views.py contains the business logic of the application. It handles user requests, processes data, interacts with models if needed, and returns responses. A view function receives an HTTP request and returns an HTTP response, such as rendering an HTML template or displaying data.

views.py contains business logic.

It handles user requests and returns responses.

Example:

```
from django.http import HttpResponse
from django.shortcuts import render

def home(request):
    return HttpResponse("Welcome to Django")

def about(request):
    return render(request, 'about.html')
```

Responsibilities:

- Receives HTTP request
- Interacts with models
- Sends data to template
- Returns HTTP response

admin.py is used to register models in the Django admin panel. By registering models in this file, we can manage database records (add, update, delete, view) through Django's built-in administrative interface without writing extra code.

apps.py contains the configuration settings for the app. It defines the app name and app-specific settings. Django uses this file to recognize and configure the application properly when it is added to `INSTALLED_APPS`.

tests.py is used to write test cases for the application. It helps developers test models, views, and other functionalities to ensure the application works correctly. Testing improves reliability and reduces errors.

The **migrations/** folder stores migration files. These files are automatically generated when changes are made in `models.py`. Migrations help in applying changes to the database structure, such as creating tables or modifying fields, using commands like `makemigrations` and `migrate`.

Creating Views

A **view** is a Python function (or class) that handles HTTP requests and returns HTTP responses.

Views contain the business logic of the application.

Views are created inside the **views.py** file of an app.

Example of Creating a View

```
from django.http import HttpResponse
from django.shortcuts import render
```

```
def home(request):
    return HttpResponse("Welcome to Django")
```

```
def about(request):
    return HttpResponse("This is About Page")
```

- **request** → Represents user request.
- **HttpResponse()** → Sends text response to browser.
- **render()** → Used to return HTML templates.

Example with template:

```
def home(request):
    return render(request, 'home.html')
```

Mapping URLs

URL mapping means connecting a URL path to a specific view function.

It is done inside **urls.py**.

Project-Level URL Mapping

File: **project/urls.py**

```
from django.contrib import admin
from django.urls import path
from myapp import views
```

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path("", views.home),
    path('about/', views.about),
]
```

- path() function maps URL to view
- "" → Home page
- 'about/' → About page
- views.home → Calls home function
- views.about → Calls about function

App-Level URL Mapping

Step 1: Create urls.py inside app

```
from django.urls import path
from . import views
```

```
urlpatterns = [
    path("", views.home),
    path('about/', views.about),
]
```

Step 2: Include App URLs in Project urls.py

```
from django.contrib import admin
from django.urls import path, include
```

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path("", include('myapp.urls')),
]
```


HTTP request-response cycle in Django,

The HTTP request-response cycle in Django is the sequence of events that handle a client's request and return a response. This process involves several components working in a specific order: the web server, WSGI interface, middleware, URL dispatcher, views, and templates/models.

The Request Phase (Client to View)

1. **Client Request:** A user initiates a request (e.g., by typing a URL in a browser, clicking a link, or submitting a form). The browser creates an `HttpRequest` object containing metadata like the HTTP method (GET, POST), URL, headers, and body data.
2. **Web Server & WSGI:** A web server (like Nginx or Apache) receives the request and forwards it to a Web Server Gateway Interface (WSGI) application server (like Gunicorn). The WSGI handler acts as a bridge, translating the request into a format the Django application can understand.
3. **Middleware (Request Processing):** The request first passes through a series of middleware components defined in your `settings.py` file. Each middleware can inspect, modify, or even short-circuit the request and return an `HttpResponse` immediately (e.g., for security checks, authentication, or session management). They are processed in the order they appear in the settings.
4. **URL Dispatcher:** If the request passes all middleware checks, Django's URL dispatcher (defined in `urls.py`) takes the URL path and attempts to match it with a defined pattern. If a match is found, the associated view function or class is identified. If no pattern matches, Django returns a 404 error.
5. **View Processing:** The matched view function is called, and the `HttpRequest` object is passed as its first argument. This is where the core business logic resides. The view can:
 1. Access data from the request object (GET/POST parameters, files, cookies).
 2. Interact with the database using Django's ORM (Models) to retrieve or save data.
 3. Perform necessary calculations or operations.

The Response Phase (View to Client)

1. **Response Generation:** The view is responsible for generating and returning an `HttpResponse` object. This response could be:
 1. Simple text or a redirect.
 2. A rendered HTML page using Django's template system, where data is passed to a template file (e.g., `render(request, 'template.html', context)`).
 3. JSON data, common in API development using Django REST Framework's `Response` class.
2. **Middleware (Response Processing):** The generated `HttpResponse` object then travels back through the same middleware components, but in reverse order. Here, middleware can perform post-processing tasks, such as adding security headers, compressing content, or logging response times.
3. **WSGI & Web Server:** The final response is passed back through the WSGI server and the web server to the client's browser.
4. **Client Display:** The client's browser receives the response, interprets the content (e.g., renders the HTML), and displays the resulting web page to the user.

Debugging with Error Pages in Django

Debugging with error pages in Django depends mainly on the value of the **DEBUG** setting in `settings.py`. Django behaves very differently in **development** and **production** environments to balance **developer convenience** and **application security**.

1. Debugging in Development Environment (DEBUG = True)

When `DEBUG = True`, Django provides **detailed error pages** whenever an error occurs. These pages are designed to help developers quickly find and fix problems.

a) Detailed Traceback

- Django displays a **full Python traceback** when an exception occurs.
- It shows:
 - The exact file name

- Line number where the error happened
- The sequence of function calls (stack trace)
- Local variable values at each level of execution are also shown, which helps in understanding *why* the error occurred.
- You can immediately identify the source of the bug.

b) Request Information

The debug page also shows:

- HTTP request details (GET, POST data, cookies)
- Current settings
- Installed apps and middleware
- Helps diagnose issues related to requests, forms, middleware, or misconfiguration.

c) Interactive Debugging Tools

- Using tools like **django-extensions** with `runserver_plus`, Django can show an **interactive Python console** in the error page.
- You can inspect variables and run Python commands directly from the browser.
- Makes debugging faster and more hands-on.

d) Using `assert False` for Debugging

- Developers sometimes place `assert False` intentionally in the code.
- This forces Django to raise an `AssertionError` and display the debug page.

Example:

```
def my_view(request):  
    x = 10  
    assert False  
    return HttpResponse("Hello")
```

Useful to inspect variable values and execution flow at a specific point.

2. Debugging in Production Environment (DEBUG = False)

In production, detailed error pages must be **disabled** to protect sensitive information.

Why must DEBUG be False?

- Debug pages expose:Source code, Database structure, Environment variables
- This can be misused by attackers.

a) Custom Error Pages

Django allows you to define **user-friendly error pages**, such as:

- 404.html – Page not found
- 500.html – Server error
- 403.html – Permission denied
- 400.html – Bad request

These pages:

- Should be simple
- Should not contain complex logic
- Should match the site's design

b) Error Reporting via Email

- Django can email error details to administrators.
- Configure the ADMINS setting in settings.py.
- When a **500 error** occurs, Django sends:
 - Full traceback
 - Request information

Developers are notified immediately when something goes wrong.

c) Logging

- Django provides a powerful **logging framework**.
- Errors can be logged to: Files, Console

Maintains a permanent record of errors for later analysis.

d) Sensitive Data Filtering

- Django filters sensitive data from error reports.
- Decorators used:
 - `@sensitive_variables()`
 - `@sensitive_post_parameters()`

Prevents passwords and confidential data from appearing in logs or emails.

Core Principles of Loose Coupling in Django

Loose coupling is a core design philosophy in Django, meaning that the different parts (components or "layers") of the framework are designed to be as independent as possible, with minimal reliance on each other's internal implementation details.

Core Principles of Loose Coupling in Django

The primary goal of loose coupling is to increase flexibility, maintainability, and testability of an application. In a loosely coupled system, a change in one component has little to no effect on the others.

Key manifestations of this principle in Django include:

- **MTV Architecture:** Django's Model-Template-View (MTV) architecture inherently separates business logic (Models), data presentation (Templates), and control flow (Views). For example, the template system doesn't need to know how the data is stored in the database, only how to display the data it receives.
- **URLconfs:** The URL definitions are separate from the view functions they call. You can change a URL path in the `urls.py` file without modifying the corresponding view function's logic, or change the view function's logic without affecting the URL that maps to it.
- **Interchangeable Components:** While Django provides a full stack of components (e.g., its own template engine and ORM), its architecture allows developers to use alternative tools if preferred, such as a different template system (like Jinja2) or a different database access layer, with minimal friction.

- **Signals:** Django's signal dispatcher allows decoupled applications to get notified when actions occur elsewhere in the framework (e.g., when a model is saved) without the sender needing to know which receivers are listening. This promotes an event-driven architecture that further reduces direct dependencies.
- **Request/Response Objects:** Views are passed an `HttpRequest` object and are responsible for returning an `HttpResponse` object. The view doesn't need to know the underlying mechanisms of how the request was processed or how the response will be finally handled, allowing for easier testing by passing in "fake" request objects.

Benefits in a Django Project

- **Easier Maintenance:** Bugs can be fixed in specific areas with less risk of introducing unintended side effects elsewhere in the application.
- **Increased Flexibility:** It's easier to adapt the application to changing requirements, such as switching from one cloud provider to another or changing a URL structure.
- **Improved Testability:** Individual components can be tested in isolation, for example, by passing a mock object instead of a real database connection, which simplifies unit testing.
- **Scalability:** Loosely coupled systems are generally more modular, making it easier to scale individual components independently as needed.

Using Virtual Environments and pip in Django

Introduction

In Django development, managing project dependencies properly is very important. Different Django projects may require different versions of Django or Python libraries. To handle this efficiently, Django developers use **virtual environments** along with **pip**, Python's package manager.

Virtual Environment

A **virtual environment** is an isolated Python environment created for a specific project. It contains its own:

- Python interpreter
- Installed packages

- Django version and dependencies

Each Django project can have its own virtual environment, independent of other projects and the system Python.

Why Virtual Environments are Needed

- Avoids version conflicts between projects
- Keeps project dependencies isolated
- Makes projects easier to maintain and deploy
- Helps in team collaboration by ensuring consistent environments

Creating a Virtual Environment

```
python -m venv venv
```

This command creates a folder named venv that contains the isolated environment.

Activating the Virtual Environment

Windows

```
venv\Scripts\activate
```

When activated, the terminal shows (venv) indicating that the virtual environment is active.

Deactivating the Virtual Environment

```
deactivate
```

This returns the system to the global Python environment.

pip Package Manager

pip is Python's default package management tool used to:

- Install packages
- Upgrade packages
- Remove packages

In Django, pip is used to install Django and all required dependencies inside the virtual environment.

Installing Django Using pip

```
pip install django
```

This installs Django only within the active virtual environment.

To check the installed Django version:

```
django-admin --version
```

Viewing Installed Packages

```
pip list
```

This command displays all packages installed in the current virtual environment.

Managing Dependencies with requirements.txt

Creating requirements.txt

```
pip freeze > requirements.txt
```

This file stores the list of installed packages and their versions.

Example:

```
Django==4.2.7  
asgiref==3.7.2  
sqlparse==0.4.4
```

Installing Dependencies from requirements.txt

```
pip install -r requirements.txt
```

This ensures that the same package versions are installed on another system.

Importance of Virtual Environments and pip in Django

- Ensures project-specific dependency management

- Prevents conflicts between different Django projects
- Makes deployment and testing easier
- Supports reproducibility across development environments

Working with Development vs Production Settings in Django

In Django, **settings.py** controls how the application behaves. Django applications are usually run in two environments:

- **Development environment**
- **Production environment**

Each environment has different requirements. Django provides settings that can be configured to suit both scenarios securely and efficiently.

1. Development Environment

The **development environment** is used while building and testing the application locally.

Purpose

- To help developers find and fix errors quickly
- To experiment and test features safely

Key Characteristics of Development Settings

DEBUG = True

- Displays detailed error pages
- Shows full traceback, line numbers, and variable values
- Useful for debugging

ALLOWED_HOSTS

ALLOWED_HOSTS = []

- Not required when DEBUG = True
- Django allows requests from localhost

Database

- Usually uses **SQLite**
- Simple, file-based, no extra setup

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.sqlite3',  
        'NAME': BASE_DIR / 'db.sqlite3',  
    }  
}
```

Static Files

- Served automatically by Django
- No extra configuration needed

Security

- Security settings are relaxed
- Suitable only for local testing

2. Production Environment

The **production environment** is used when the application is deployed for real users.

Purpose

- To serve end users safely
- To protect sensitive data
- To ensure performance and reliability

Key Characteristics of Production Settings

DEBUG = False

DEBUG = False

- Disables detailed error pages
- Prevents exposure of source code and sensitive data
- Shows generic error pages (404, 500)

ALLOWED_HOSTS

ALLOWED_HOSTS = ['example.com', 'www.example.com']

- Required when DEBUG = False
- Prevents HTTP Host header attacks

Custom Error Pages

- User-friendly error pages:
 - 404.html
 - 500.html
 - 403.html
 - 400.html

Database

- Uses production-grade databases:
 - PostgreSQL
 - MySQL
 - Oracle

Static Files

- Served by web servers like:
 - Nginx
 - Apache
- Collected using:

```
python manage.py collectstatic
```

Security Settings

Enabled for safety:

```
SECURE_SSL_REDIRECT = True
```

```
CSRF_COOKIE_SECURE = True
```

```
SESSION_COOKIE_SECURE = True
```

Logging and Error Reporting

- Errors are logged to files or monitoring tools
- Admins can receive error emails

```
ADMINS = [('Admin', 'admin@example.com')]
```